
Fine-Tuning Open-Weight Language Models

A technical course on data, LoRA/QLoRA, evaluation, and deployment

21 August 2026

- Course overview
 - What you will produce
- 1. Decide whether fine-tuning is the right tool
 - 1.1 Start with a measurable task
 - 1.2 Use the least invasive adaptation that works
 - 1.3 Select an open-weight foundation model
- 2. Engineer the adaptation dataset
 - 2.1 Define the example schema
 - 2.2 Build splits that measure generalization
 - 2.3 Quality controls
- 3. Understand LoRA and QLoRA
 - 3.1 Core configuration choices
 - 3.2 What QLoRA changes
 - 3.3 A reviewable experiment configuration
- 4. Run a reproducible training experiment
 - 4.1 Preflight
 - 4.2 Training loop seam
 - 4.3 Read the curves
- 5. Evaluate the model and the whole system
 - 5.1 Evaluation matrix
 - 5.2 Compare like with like
 - 5.3 Predefine stop conditions
- 6. Package, deploy, and monitor
 - 6.1 Release bundle
 - 6.2 Serving decisions
 - 6.3 Monitor what can drift
- Capstone: technical adaptation plan
- Technical checklist
 - Before training
 - Before release
- Selected sources

Course overview

This course is a practical guide to adapting an open-weight language model for a bounded task. It deliberately focuses on the engineering decisions that make a fine-tuning experiment reproducible and useful: choosing a base model, proving that tuning is necessary, constructing clean datasets, configuring LoRA or QLoRA, evaluating against a baseline, and packaging the result for controlled deployment.

The course is framework-aware but not tied to a single training stack. Examples use Hugging Face-style names because they make the moving parts concrete.

Audience: ML engineers, data scientists, and software engineers with basic familiarity with transformer-based language models.

Level: Intermediate.

Intended outcome: Design a reproducible LoRA or QLoRA experiment, evaluate it against untuned and retrieval-augmented baselines, and produce an evidence-based ship, revise, or stop decision.

Estimated learning time: 6 hours.

What you will produce

By the end of the course you will have:

1. a base-model selection record;
2. a decision record comparing prompting, RAG, PEFT, and full fine-tuning;
3. versioned train, validation, and test datasets;
4. a LoRA or QLoRA experiment configuration;
5. an evaluation report with quality, safety, and operational metrics;
6. a model card addendum and deployment/rollback plan.

1. Decide whether fine-tuning is the right tool

Fine-tuning changes model behavior by updating parameters. That makes it useful for persistent behavior—format, style, task procedure, domain-specific patterns, or tool-use conventions—but a poor default for facts that change frequently or must be cited.

1.1 Start with a measurable task

Write the task as an input/output contract before choosing a model or training method.

Contract field	Example
Input	A support request plus allowed product metadata
Output	A JSON object conforming to a fixed schema
Success metric	At least 95% schema-valid outputs and macro F1 above 0.88
Latency budget	p95 below 1.5 seconds
Failure behavior	Abstain when required context is missing
Out of scope	Generating facts not present in the input or retrieval context

The baseline must run on the same test set as every later experiment. Record the prompt, sampling parameters, model checkpoint, inference runtime, and hardware.

1.2 Use the least invasive adaptation that works

Evaluate approaches in increasing order of cost and operational coupling:

1. **Prompting:** best when the base model already has the capability and only needs clearer instructions or examples.
2. **Retrieval-augmented generation (RAG):** best when the missing element is changing, private, attributable knowledge.
3. **Parameter-efficient fine-tuning (PEFT):** best when behavior remains unreliable after prompt and context design, and a small adapter can encode the desired pattern.
4. **Full fine-tuning:** justified only when PEFT cannot reach the target and the expected gain warrants much higher compute, storage, and operational cost.

Do not fine-tune simply because a model sometimes answers incorrectly. First classify the failure:

Failure	Preferred first response
Missing or stale knowledge	RAG
Inconsistent output format	Constrained decoding, schema validation, then PEFT
Weak task procedure	Prompt examples, then PEFT
Context omitted by the application	Fix the application or retrieval pipeline
Safety boundary failure	Guardrails, data changes, safety tuning, and evaluation
Base capability absent	Try a stronger base model before training

1.3 Select an open-weight foundation model

“Open-weight” does not automatically mean “Open Source AI.” Inspect the license and model documentation directly. Record:

- immutable model identifier and revision;
- license, acceptable-use restrictions, redistribution rights, and notice obligations;
- parameter count, architecture, context length, and tokenizer;
- supported languages and task fit;
- memory needed for training and inference;
- precision and quantization support;
- model-card limitations and known evaluation results;
- framework and serving-runtime compatibility.

Exercise — model gate: Compare two candidate checkpoints using a one-page table. Reject any candidate whose license, memory requirement, or task capability violates a hard constraint.

2. Engineer the adaptation dataset

The dataset is executable task specification. A small, representative, well-reviewed dataset is often more valuable than a large collection of noisy examples.

2.1 Define the example schema

Use one canonical representation and convert it to the framework-specific chat template only during preprocessing.

```
{
  "example_id": "support-004218",
  "task_version": "2.1",
  "system": "Return only JSON matching the supplied schema.",
  "input": "Customer text and permitted context go here.",
  "output": "{\"category\":\"billing\",\"confidence\":0.93}",
  "slice": ["short-input", "billing"],
  "provenance": "reviewed-synthetic-v3",
  "license": "internal-approved",
  "created_at": "2026-08-01"
}
```

Keep metadata outside the text presented to the model unless it is part of the real inference contract.

2.2 Build splits that measure generalization

Split by the unit that can leak, not merely by individual row. If many rows come from the same document, user, template, or event, keep that entire group in one split.

- **Train:** used by the optimizer.
- **Validation:** used for checkpoint and hyperparameter selection.
- **Test:** frozen before training and used only for the final comparison.
- **Challenge set:** rare, adversarial, boundary, and abstention cases.

Run exact and near-duplicate detection across all splits. Freeze the test set checksum and prevent training jobs from reading it.

2.3 Quality controls

For every example, validate:

- schema and encoding;
- source and usage rights;
- absence of secrets and unnecessary personal data;

- label correctness and reviewer agreement;
- realistic input length and class distribution;
- consistent formatting and chat roles;
- no hidden answer in metadata or retrieval context;
- explicit representation of negative and abstention cases.

Version the dataset and preprocessing code together. A training run should record the raw-data snapshot, transformation commit, final dataset checksum, tokenizer revision, and filtering statistics.

Exercise — dataset card: Document intended use, collection method, schema, split strategy, filters, slice distribution, known gaps, and deletion procedure. Then manually review a stratified sample from every split.

3. Understand LoRA and QLoRA

Low-Rank Adaptation freezes the base model and trains small matrices that approximate the update to selected weight matrices.

For a frozen weight matrix W_0 , LoRA learns:

$$W = W_0 + \Delta W, \quad \Delta W = \frac{\alpha}{r} BA$$

where rank r controls adapter capacity and α scales the update. The base weights stay unchanged; the adapter can be stored, swapped, or merged separately.

3.1 Core configuration choices

Parameter	What it controls	Practical starting point
<code>r</code>	Adapter rank/capacity	8–32; increase only with evidence of underfitting
<code>lora_alpha</code>	Update scaling	Often 1–2 times <code>r</code>
<code>lora_dropout</code>	Regularization	0–0.1 depending on dataset size
<code>target_modules</code>	Layers receiving adapters	Start with attention projections; test broader coverage
<code>learning rate</code>	Optimizer step size	Usually higher than full fine-tuning; sweep logarithmically
<code>epochs/steps</code>	Exposure to training data	Select using validation curves, not a fixed habit

The correct target-module names depend on the architecture. Inspect the model rather than copying a configuration from a different family.

3.2 What QLoRA changes

QLoRA loads the frozen base model in low-bit precision while training LoRA adapters at a higher compute precision. It reduces memory use, enabling larger models on constrained hardware, but introduces more moving parts:

- quantization type and bit width;
- compute dtype and hardware support;
- quantization library compatibility;
- possible quality or throughput tradeoffs;

- stricter reproducibility requirements across drivers and kernels.

Use QLoRA when memory is the binding constraint. Do not assume it is automatically faster.

3.3 A reviewable experiment configuration

```
experiment_id: task-v2-gemma-lora-r16-seed42
base_model:
  id: organization/model-name
  revision: immutable-commit-or-tag
  license_review: approved-record-id
data:
  dataset_version: task-v2.3
  train_checksum: sha256:...
  validation_checksum: sha256:...
  max_sequence_length: 2048
adapter:
  method: lora
  rank: 16
  alpha: 32
  dropout: 0.05
  target_modules: [q_proj, k_proj, v_proj, o_proj]
training:
  seed: 42
  learning_rate: 0.0002
  warmup_ratio: 0.03
  effective_batch_size: 64
  epochs: 3
  precision: bf16
evaluation:
  primary_metric: macro_f1
  select_checkpoint_by: validation_macro_f1
  compare_to: [untuned-zero-shot, untuned-few-shot, rag-baseline]
```

Treat this as a starting hypothesis, not a universal recipe.

4. Run a reproducible training experiment

4.1 Preflight

Before allocating training compute:

1. validate a small batch end to end;
2. render the exact tokenized prompts and labels;
3. verify that padding tokens are excluded from loss;
4. inspect truncation rates and sequence-length distribution;
5. overfit a tiny sample to confirm that gradients and labels work;
6. run the untuned baseline and save predictions;
7. estimate peak memory and wall-clock time.

Failure to overfit a tiny clean sample usually indicates a pipeline or configuration defect. Successful tiny-sample overfitting does not prove generalization.

4.2 Training loop seam

Keep data preparation, model construction, training, and evaluation independently testable.

```
seed_everything(config.training.seed)
dataset = load_versioned_dataset(config.data)
tokenized = preprocess(dataset, tokenizer_revision=config.base_model.revision)

base_model = load_base_model(
    model_id=config.base_model.id,
    revision=config.base_model.revision,
    quantization=config.adapter.get("quantization"),
)
model = attach_lora(base_model, config.adapter)

trainer = build_trainer(
    model=model,
    train_data=tokenized.train,
    validation_data=tokenized.validation,
    training_config=config.training,
)
trainer.train()
save_adapter_and_run_manifest(model, config, metrics=trainer.metrics)
```

The run manifest should capture code commit, environment lockfile, model and data revisions, random seeds, hardware, driver/runtime versions, configuration, checkpoints, metrics, and artifact checksums.

4.3 Read the curves

Track training loss and task metrics on validation slices. Warning patterns include:

- training loss falls while validation quality degrades: overfitting or distribution mismatch;
- both curves stall early: insufficient capacity, weak labels, bad learning rate, or unsuitable base model;
- average metric improves while a critical slice regresses: dataset imbalance or harmful tradeoff;
- format validity improves but semantic accuracy does not: the adapter learned presentation, not the task.

Run a small, documented sweep over the parameters most likely to matter. Do not select a checkpoint by repeatedly inspecting the test set.

5. Evaluate the model and the whole system

Fine-tuning is successful only if it improves the deployed system against the agreed baseline without unacceptable regressions.

5.1 Evaluation matrix

Dimension	Example measures
Task quality	accuracy, precision/recall/F1, exact match, rubric score
Structured output	schema validity, required-field completeness
Grounding	citation correctness, claim support, retrieval relevance
Robustness	paraphrases, typos, long inputs, missing context, distribution shifts
Safety	leakage, harmful completion rate, prompt-injection success, refusal quality
Fairness	metric deltas across relevant slices
Operations	p50/p95 latency, tokens per second, peak memory, cost per request
Reliability	timeout rate, invalid response rate, fallback and abstention rate

Report confidence intervals or repeated-run variation where practical. For generative tasks, combine automated measures with blind human review using an explicit rubric. If an LLM acts as judge, calibrate it against human labels and do not treat it as independent ground truth.

5.2 Compare like with like

Every candidate must use:

- the same frozen test set;
- equivalent decoding settings where applicable;
- the same retrieval corpus and filters for RAG comparisons;
- the same output parser and failure policy;
- the same hardware class for latency comparisons;
- a documented threshold chosen before final test execution.

A useful result table includes both aggregate and slice-level metrics:

Candidate	Primary quality	Invalid output	Critical-slice score	p95 latency	Decision
Untuned + prompt	—	—	—	—	Baseline
Untuned + RAG	—	—	—	—	Compare
LoRA adapter	—	—	—	—	Compare
QLoRA adapter	—	—	—	—	Compare

5.3 Predefine stop conditions

Abandon or redesign weight adaptation if:

- it does not beat the strongest simpler baseline by the required margin;
- gains disappear on challenge or temporal-holdout sets;
- critical safety or privacy regressions appear;
- required hardware, latency, or licensing constraints are violated;
- results cannot be reproduced from the recorded artifacts;
- the target behavior can be delivered more reliably through deterministic code or retrieval.

6. Package, deploy, and monitor

Keep the base model, adapter, tokenizer, prompt template, retrieval configuration, and inference code as separately versioned artifacts. Never identify a deployment only as “the fine-tuned model.”

6.1 Release bundle

A release candidate should contain:

- base-model identifier, immutable revision, and license record;
- adapter weights and checksum;
- tokenizer and chat-template revisions;
- training configuration and run manifest;
- dataset-card and dataset-version references;
- evaluation report with baseline comparison;
- model-card addendum describing intended use and limitations;
- inference configuration, resource requirements, and dependency lockfile;
- rollback target and compatibility notes.

6.2 Serving decisions

Adapters can remain separate or be merged into base weights. Separate adapters simplify swapping and provenance; merged weights can simplify serving and sometimes improve runtime behavior. Benchmark the actual serving stack before deciding.

Use a staged rollout:

1. offline evaluation;
2. shadow traffic with no user-visible effect;
3. limited cohort or canary;
4. progressive exposure with monitored thresholds;
5. rollback if a stop condition fires.

6.3 Monitor what can drift

Track input length and category distribution, retrieval quality, output validity, abstention, user corrections, safety signals, latency, resource use, and task-quality samples. Connect each alert to an owner and response playbook.

Fine-tuning does not freeze the surrounding system. Prompt templates, retrieval indexes, inference libraries, quantization kernels, and upstream data can all change behavior even when adapter weights remain identical.

Capstone: technical adaptation plan

Create a compact, reviewable plan for one bounded task. It must include:

1. task input/output contract and strongest untuned baseline;
2. selection record for the base checkpoint and license;
3. prompting vs RAG vs PEFT vs full-tuning decision;
4. dataset card, split design, checksums, and leakage controls;
5. LoRA/QLoRA configuration with resource estimate;
6. training run manifest and reproducibility strategy;
7. evaluation matrix, thresholds, slices, and stop conditions;
8. release bundle, staged rollout, monitoring, and rollback.

The final decision must be one of: **ship the adapter**, **revise the experiment**, **use a simpler baseline**, or **stop**. Support the decision with evidence rather than preference.

Technical checklist

Before training

- ☐ The task and failure behavior are explicit.
- ☐ Untuned prompt and RAG baselines exist.
- ☐ The base revision and license are recorded.
- ☐ The train/validation/test boundary is frozen and leakage-checked.
- ☐ Dataset provenance and deletion paths are known.
- ☐ Tokenization, masking, truncation, and tiny-sample overfit tests pass.
- ☐ Compute, storage, time, and abort budgets are approved.

Before release

- ☐ The selected checkpoint beats the strongest baseline on the frozen test set.
- ☐ Critical slices, challenge cases, and safety tests meet thresholds.
- ☐ Latency and memory fit the serving environment.
- ☐ The run can be reproduced from immutable artifacts.
- ☐ The release bundle and model-card addendum are complete.
- ☐ Monitoring, staged rollout, and rollback are tested.

Selected sources

These sources come from the previously reviewed Course and are retained because they directly support the technical learning path.

1. Open Source Initiative. [The Open Source AI Definition – 1.0.](#)
2. Google AI for Developers. [Gemma model fine-tuning.](#)
3. Google AI for Developers. [Align your models.](#)
4. Patrick Lewis et al. [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.](#)
5. Hugging Face. [LoRA.](#)
6. National Institute of Standards and Technology. [Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile.](#)
7. Google AI for Developers. [Evaluate model and system for safety.](#)